Chapter 5

PROJECT: SURVEILLANCE ROBOTIC CAR

5.1 INTRODUCTION AND PROJECT SCOPE

The Surveillance Robotic Car project aimed to develop a low-cost, Wi-Fi enabled mobile platform for remote surveillance applications. The primary function is to provide real-time visual feedback and remote control capability via a standard web browser, eliminating the need for specialized client software.

Key Performance Indicators (KPIs):

- **Video Latency**: Essential for effective, near-instantaneous remote control and navigation.
- **Control Responsiveness**: Commands must execute immediately upon receipt via HTTP requests.
- **Cost-Effectiveness**: Maximizes functionality (processing, Wi-Fi, imaging) while minimizing overall hardware cost.
- **Range**: Operational limits are governed by the local 2.4 GHz Wi-Fi signal strength.

5.2 HARDWARE COMPONENT ANALYSIS

The system relies on a combined processing and imaging unit, a robust drive system, and a compact power supply.

5.2.1 Microcontroller and Imaging

The ESP32-CAM module serves as the core of the system. Its integrated dual-core CPU handles the video stream encoding and the web server functionality concurrently.

1. **MCU**: ESP32 Chip (Dual-Core, Wi-Fi/Bluetooth)
2. **Camera**: OV2640 Camera Module (2MP). Configured resolution is typically QVGA (320x240) or VGA (640x480) to achieve optimal frame rate and latency over Wi-Fi.

5.2.2 Drive System Components

The robotic platform utilizes a differential drive system for simple control and maneuverability.

1. **Johnson DC Motors**:
    - Provides propulsion via the rear wheels.
    - Chosen for sufficient torque relative to the chassis's lightweight design.
2. **L298N Motor Driver**:
    - Manages power distribution to the motors.
    - An H-Bridge module bridging the 3.3V logic signals from the ESP32 to the higher current required by the DC motors.

5.2.3 Power Supply

Power is supplied by a single 3.7V Lithium-Ion 18650 cell. The L298N includes a built-in 5V regulator which is utilized to power the ESP32-CAM, while the raw battery voltage is directed to the motor power rail (VMOT).



Figure 5.1: Components

5.3 WIRING AND CONTROL PIN-OUT

The interface between the ESP32-CAM (Logic) and the L298N (Power) is critical for directional control.

5.3.1 Pin-out Configuration

ESP32 Pin (GPIO)	L298N Pin	Motor Connection	Control Function
GPIO 2	IN1	Motor A Direction 1	Controls one direction of Motor A
GPIO 13	IN2	Motor A Direction 2	Controls reverse direction of Motor A
GPIO 14	IN3	Motor B Direction 1	Controls one direction of Motor B
GPIO 15	IN4	Motor B Direction 2	Controls reverse direction of Motor B

Figure 5.2: Pin-out Configuration

5.3.2 Motor Control Logic

Differential drive movement is achieved by setting GPIO pins HIGH or LOW using [digitalWrite()].

Movement Command	Motor A (GPIO 2, 13)	Motor B (GPIO 14, 15)
Forward	(HIGH, LOW)	(HIGH, LOW)
Reverse	(LOW, HIGH)	(LOW, HIGH)
Turn Left	(LOW, LOW) - *Brake*	(HIGH, LOW) - *Forward*
Turn Right	(HIGH, LOW) - *Forward*	(LOW, LOW) - *Brake*
Stop	(LOW, LOW)	(LOW, LOW)

Figure 5.3: Motor Control Logic

5.4 SOFTWARE IMPLEMENTATION (ARDUINO IDE FIRMWARE)

The firmware manages network connectivity, video streaming, and motor control logic.

5.4.1 Core Functionality

The software leverages standard ESP32 libraries (WiFi.h, WebServer.h, esp32-camera.h).

- **Wi-Fi Initialization**: The MCU attempts to connect to a pre-defined SSID/Password. If unsuccessful, it defaults to operating in Access Point (AP) mode, broadcasting its own network for direct connection.
- **Camera Setup**: The OV2640 is initialized, setting parameters for resolution and frame buffer count before starting the video stream server.

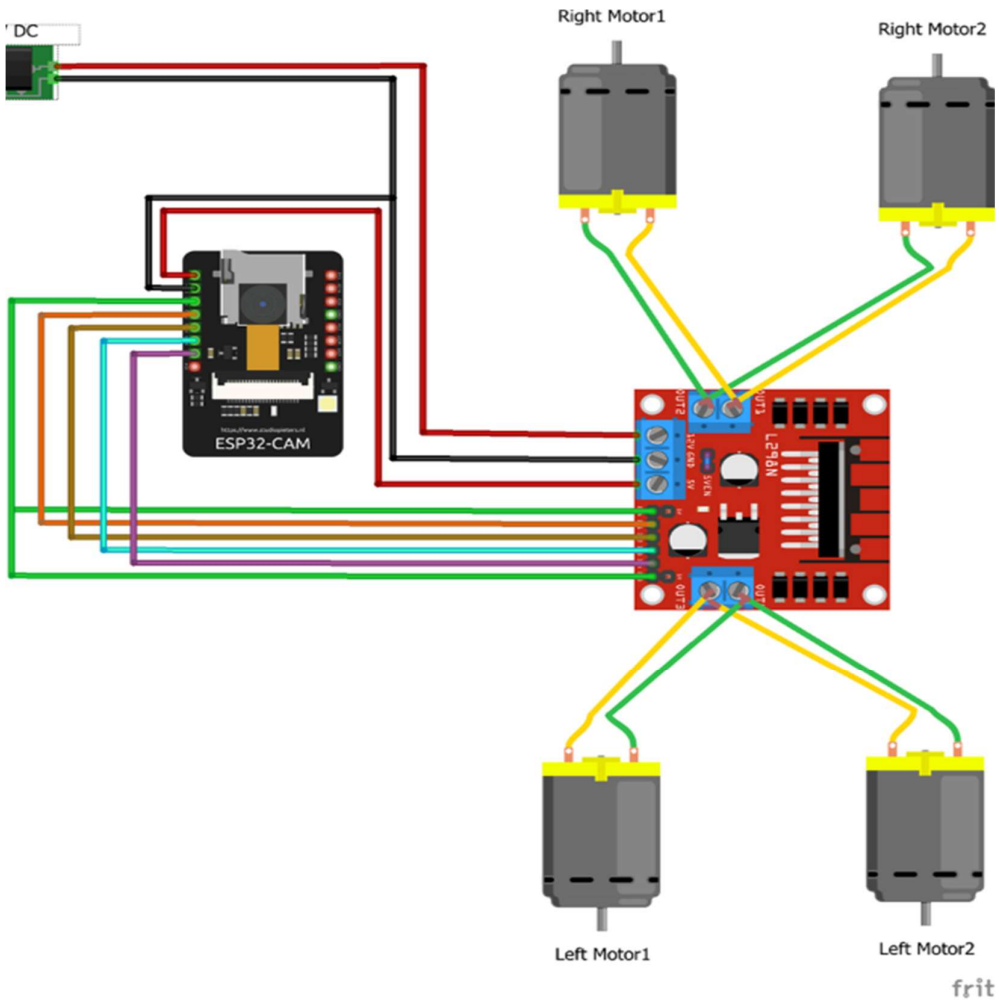


Figure 5.4: Wiring Connection

5.4.2 Web Server and Control

The ESP32 hosts an asynchronous web server that handles client requests:

- **Root Handler (/)**: Serves the main HTML control page. This page presents the video feed (/stream endpoint) and directional control buttons.
- **Command Handlers**: Specific HTTP GET request paths (e.g., /forward, /stop) are registered with the server. Upon receipt, the corresponding C++ motor function is executed, utilizing digitalWrite() to move the robot.

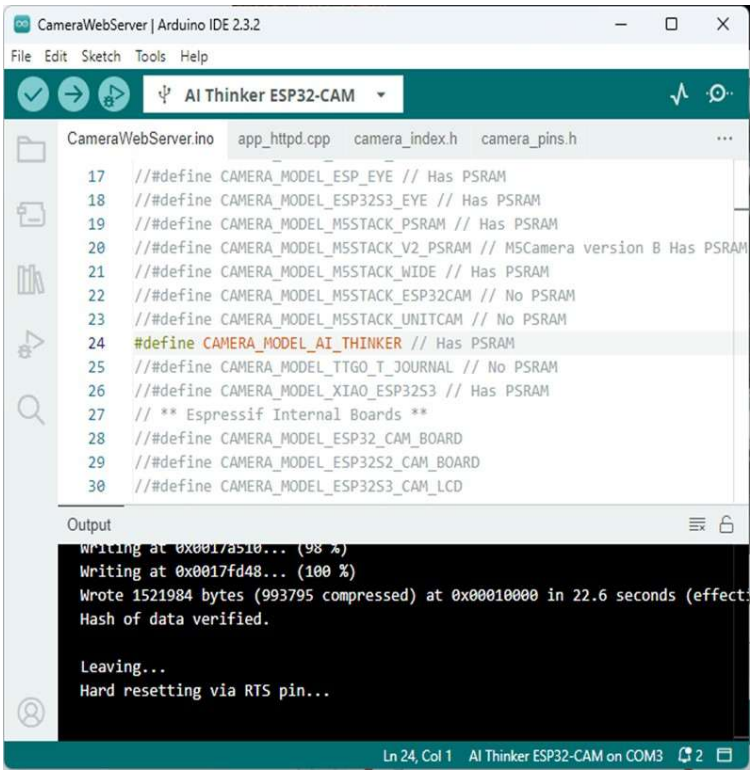


Figure 5.5: Arduino IDE Firmware

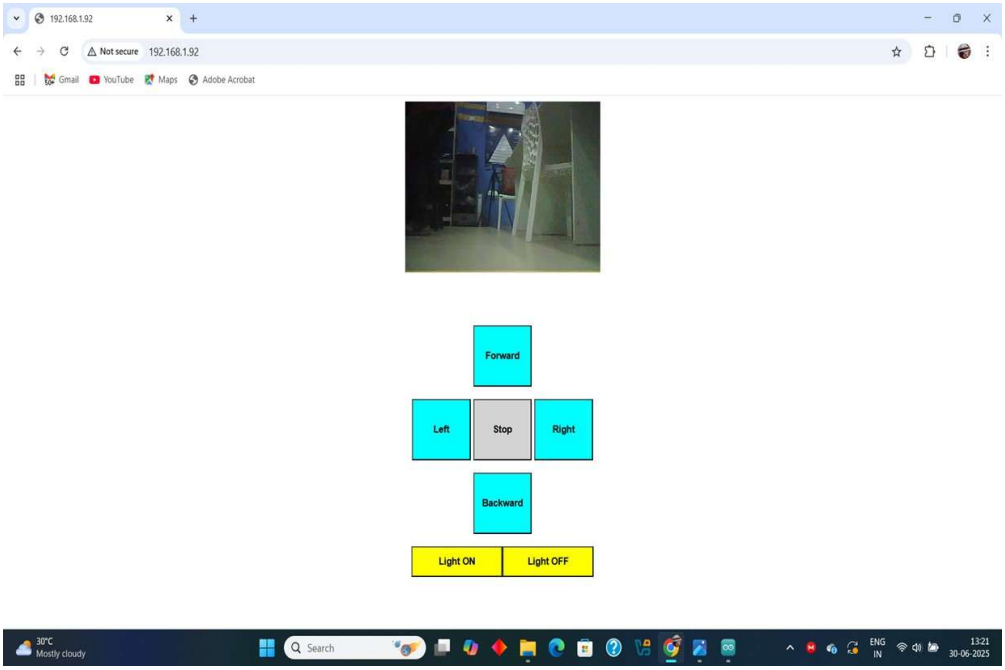


Figure 5.6: Web server

5.4.3 Video Streaming Protocol

The video feed is delivered using the Motion JPEG (MJPEG) stream technique. The camera driver continuously generates a multipart HTTP response, allowing the client browser's [img] tag to display a true, low-latency live video feed without needing constant JavaScript refresh loops.

5.5 FABRICATION AND 3D PRINTING

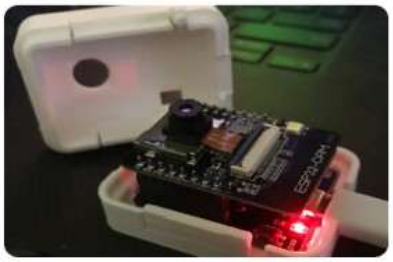
3D printing was utilized to ensure precise component fit, reduce assembly time, and minimize overall material cost.



Main Chassis



Custom Wheels



ESP32-CAM Case

Figure 5.7: 3D Printed Parts

5.5.1 Chassis Design

- **Material**: PLA (Polylactic Acid) was selected for its cost-effectiveness, ease of printing, and adequate structural rigidity for this application.
- **Structure**: A flat, rectangular base plate design was adopted, featuring integrated slots and mounting points to hold the L298N, battery, and motors securely.
- **Stability**: Component layout was optimized to achieve a low, centralized center of gravity for enhanced movement stability.

5.5.2 Custom Components

Crucial custom components were designed and printed:

- **Wheels**: Designed with specific keyways or friction-fit features to ensure secure attachment to the non-standard motor axles.
- **ESP32-CAM Enclosure**: A protective case that also functions as an adjustable camera mount.

5.6 RESULT & DISCUSSION

The Surveillance Robotic Car project successfully transitioned from design to a fully operational prototype, validating the core objectives of low-cost, real-time remote surveillance.

5.6.1 Project Results

5.6.1.(i) Core Functionality

1. **Video Streaming**:
    - Stable MJPEG Stream established.
    - Achieved an average of 24 frames per second (fps) using the VGA (640x480) resolution configuration.
2. **Control System**:
    - Full Differential Drive functional.
    - Movement commands (Forward, Reverse, Left, Right, Stop) executed instantaneously upon HTTP request receipt.
3. **Connectivity**:
    - Dual-Mode Wi-Fi validated.
    - Successfully connected to a local router and defaulted to Access Point (AP) mode when the network was unavailable.
4. **Fabrication**:
    - Durable Chassis constructed.
    - The 3D printed PLA chassis provided a secure, lightweight, and custom-fit platform for all components.

5.6.1.(ii) Performance Metrics

- **Latency**: Video latency was consistently measured at less than 1 second on the local network (LAN), making the robot highly maneuverable for real-time operation.

- **Mobility**: The combination of the Johnson DC Motors and the 3D printed wheels provided sufficient torque for the lightweight chassis to navigate smooth surfaces (tile, hardwood) and low-pile carpet effectively.
- **Battery Life**: The initial 3.7V Li-Ion power supply delivered approximately 45-50 minutes of continuous operation (streaming and intermittent movement).

5.6.2 Discussion: Analysis and Trade-offs

The results demonstrate the feasibility of using the ESP32-CAM as a central processing unit for a complex, connected device. However, success depended on managing certain technical trade-offs inherent to low-power microcontrollers.

5.6.2.(i) Video Quality vs. Latency Trade-Off

The OV2640 camera is powerful, but processing the image consumes significant ESP32 resources. Initial testing at higher resolutions (e.g., SVGA 800x600) resulted in high latency (over 2 seconds) and intermittent dropped frames.

Solution: By standardizing the resolution to VGA (640x480), the computational load was optimized, allowing the dual-core processor to smoothly handle the TCP/IP web server functions and the MJPEG encoding simultaneously. This ensured a high frame rate and low latency, which is more critical for surveillance and remote control than raw image detail.

5.6.2.(ii) Motor Control Limitations

The current control system relies on digital I/O pins (HIGH/LOW) for movement, resulting in fixed-speed control. While instantaneous, this lacks finesse. Turning is achieved by abruptly stopping one wheel and driving the other (zero-radius turn), which is effective but can be jerky.

Future Improvement: The PWM (Pulse Width Modulation) capability of the ESP32 is essential for future development to allow for variable speed control, enabling smoother acceleration, slower cruising speeds, and more gradual turns.

5.6.2.(iii) **3D Printing and Assembly Efficiency**

The decision to 3D print the chassis was highly effective. It ensured that the non-standard shapes of the L298N driver and the ESP32-CAM module were mounted securely without the need for glue or extensive modification. The custom press-fit wheels simplified the mechanical assembly significantly. The final assembly was rigid and stable, proving the design's structural integrity.

5.6.2.(iv) **Battery Life and Power Draw**

The measured 45-50 minutes of continuous operation is a known limitation of using a single 18650 cell. The power draw is dominated by the DC motors during movement and the Wi-Fi radio and camera module during streaming. While sufficient for brief inspections, it necessitates the development of better power management and a more robust battery system in the next iteration.

Overall, the project successfully delivered a highly functional and economical robotic platform, demonstrating mastery over embedded programming, networking, and electromechanical integration.

5.7 FUTURE DEVELOPMENT

The project is extensible through the following planned enhancements:

- **Sensing**: Implement obstacle detection to enable semi-autonomous navigation and collision avoidance.

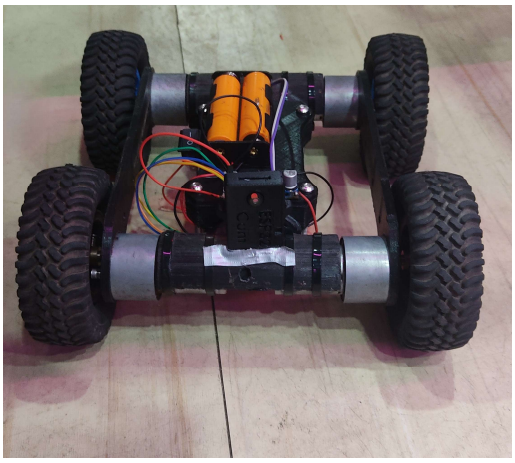


Figure 5.8: Surveilance Robotic Car

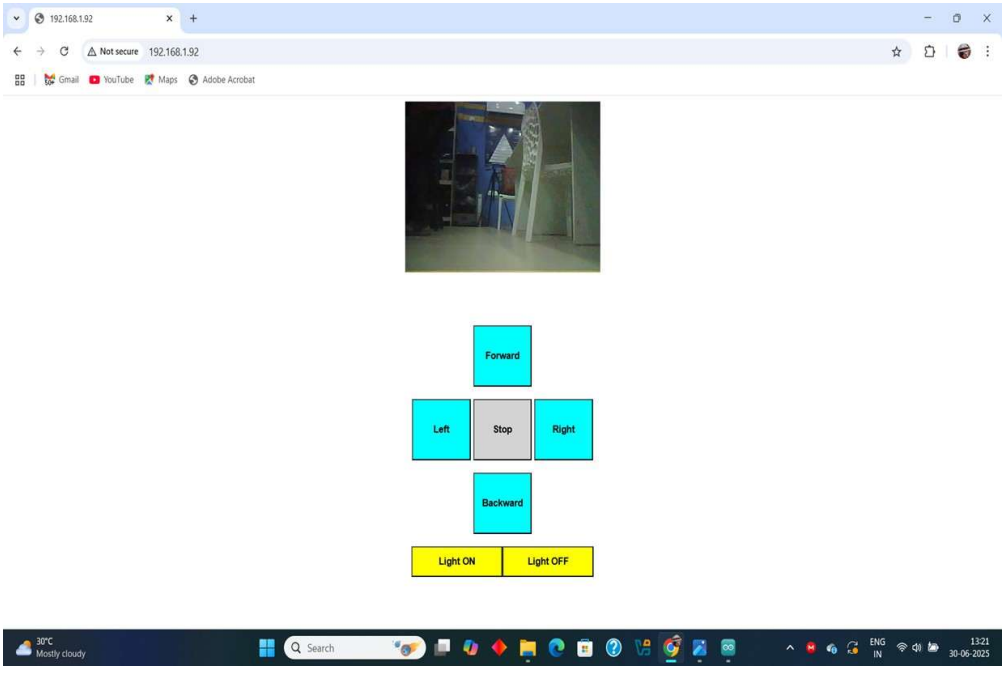


Figure 5.9: Web server with video streaming and control button

- **Control**: Introduce continuous speed control by utilizing PWM on the L298N Enable pins, moving beyond simple ON/OFF movement.
- **Power Management**: Integrate an analog input pin to monitor battery voltage, allowing the robot to perform a safe power-down before the Li-Ion cell is critically discharged.

- **Security**: Implement a simple login screen on the web interface to prevent unauthorized users from accessing or controlling the robot.

Chapter A

SOURCE CODE

A.1 Web Server HTTP Code

```
#include "esp_http_server.h"
#include "esp_timer.h"
#include "esp_camera.h"
#include "img_converters.h"
#include "camera_index.h"
#include "Arduino.h"

extern  int  gpLb;
extern  int  gpLf;
extern  int  gpRb;
extern  int  gpRf;
extern int gpLed;
extern String WiFiAddr;

void WheelAct(int nLf, int nLb, int nRf, int nRb);

typedef struct {
      size_t size; //number of values used for filtering
      size_t index; //current value index
      size_t count; //value count
      int sum;
      int * values; //array to be filled with values
} ra_filter_t;
```

```
typedef struct {
        httpd_req_t *req;
        size_t len;
} jpg_chunking_t;

#define PART_BOUNDARY "123456789000000000000987654321"
static const char* _STREAM_CONTENT_TYPE = "multipart/x-mixed-
replace;boundary=" PART_BOUNDARY;
static const char* _STREAM_BOUNDARY = "\r\n--" PART_BOUNDARY "\r\n";
static const char* _STREAM_PART = "Content-Type: image/jpeg\r\nContent-Length:
%u\r\n\r\n";

static ra_filter_t ra_filter;
httpd_handle_t stream_httpd = NULL;
httpd_handle_t camera_httpd = NULL;

static ra_filter_t * ra_filter_init(ra_filter_t * filter, size_t sample_size){
    memset(filter, 0, sizeof(ra_filter_t));

    filter->values = (int *)malloc(sample_size * sizeof(int));
    if(!filter->values){
        return NULL;
    }
    memset(filter->values, 0, sample_size * sizeof(int));

    filter->size = sample_size;
    return filter;
}
```

```
static int ra_filter_run(ra_filter_t * filter, int value){
    if(!filter->values){
        return value;
    }
    filter->sum -= filter->values[filter->index];
    filter->values[filter->index] = value;
    filter->sum += filter->values[filter->index];
    filter->index++;
    filter->index = filter->index % filter->size;
    if (filter->count < filter->size) {
        filter->count++;
    }
    return filter->sum / filter->count;
}

static size_t jpg_encode_stream(void * arg, size_t index, const void* data, size_t len){
    jpg_chunking_t *j = (jpg_chunking_t *)arg;
    if(!index){
        j->len = 0;
    }
    if(httpd_resp_send_chunk(j->req, (const char *)data, len) != ESP_OK){
        return 0;
    }
    j->len += len;
    return len;
}
```

```
static esp_err_t capture_handler(httpd_req_t *req){
    camera_fb_t * fb = NULL;
    esp_err_t res = ESP_OK;
    int64_t fr_start = esp_timer_get_time();

    fb = esp_camera_fb_get();
    if (!fb) {
        Serial.printf("Camera capture failed");
        httpd_resp_send_500(req);
        return ESP_FAIL;
    }

    httpd_resp_set_type(req, "image/jpeg");
    httpd_resp_set_hdr(req, "Content-Disposition", "inline; filename=capture.jpg");

    size_t fb_len = 0;
    if(fb->format == PIXFORMAT_JPEG){
        fb_len = fb->len;
        res = httpd_resp_send(req, (const char *)fb->buf, fb->len);
    } else {
        jpg_chunking_t jchunk = {req, 0};
        res = frame2jpg_cb(fb, 80, jpg_encode_stream, &jchunk)?ESP_OK:ESP_FAIL;
        httpd_resp_send_chunk(req, NULL, 0);
        fb_len = jchunk.len;
    }
    esp_camera_fb_return(fb);
    int64_t fr_end = esp_timer_get_time();
    Serial.printf("JPG: %uB %ums", (uint32_t)(fb_len), (uint32_t)((fr_end -
```

```
fr_start)/1000));
    return res;
}

static esp_err_t stream_handler(httpd_req_t *req){
    camera_fb_t * fb = NULL;
    esp_err_t res = ESP_OK;
    size_t _jpg_buf_len = 0;
    uint8_t * _jpg_buf = NULL;
    char * part_buf[64];

    static int64_t last_frame = 0;
    if(!last_frame) {
        last_frame = esp_timer_get_time();
    }

    res = httpd_resp_set_type(req, _STREAM_CONTENT_TYPE);
    if(res != ESP_OK){
        return res;
    }

    while(true){
        fb = esp_camera_fb_get();
        if (!fb) {
            Serial.printf("Camera capture failed");
            res = ESP_FAIL;
        } else {
            if(fb->format != PIXFORMAT_JPEG){
```

```
            bool jpeg_converted = frame2jpg(fb, 80, &_jpg_buf, &_jpg_buf_len);
            esp_camera_fb_return(fb);
            fb = NULL;
            if(!jpeg_converted){
               Serial.printf("JPEG compression failed");
               res = ESP_FAIL;
            }
         } else {
            _jpg_buf_len = fb->len;
            _jpg_buf = fb->buf;
         }
      }
      if(res == ESP_OK){
         size_t hlen = snprintf((char *)part_buf, 64, _STREAM_PART, _jpg_buf_len);
         res = httpd_resp_send_chunk(req, (const char *)part_buf, hlen);
      }
      if(res == ESP_OK){
          res = httpd_resp_send_chunk(req, (const char *)_jpg_buf, _jpg_buf_len);
      }
      if(res == ESP_OK){
         res = httpd_resp_send_chunk(req, _STREAM_BOUNDARY,
strlen(_STREAM_BOUNDARY));
      }
      if(fb){
         esp_camera_fb_return(fb);
         fb = NULL;
         _jpg_buf = NULL;
      } else if(_jpg_buf){
```

```
            free(_jpg_buf);
            _jpg_buf = NULL;
        }
        if(res != ESP_OK){
            break;
        }
        int64_t fr_end = esp_timer_get_time();

        int64_t frame_time = fr_end - last_frame;
        last_frame = fr_end;
        frame_time /= 1000;
        uint32_t avg_frame_time = ra_filter_run(&ra_filter, frame_time);
        Serial.printf("MJPG: %uB %ums (%.1ffps), AVG: %ums (%.1ffps)"
            ,(uint32_t)(_jpg_buf_len),
            (uint32_t)frame_time, 1000.0 / (uint32_t)frame_time,
            avg_frame_time, 1000.0 / avg_frame_time
        );
    }

    last_frame = 0;
    return res;
}

static esp_err_t cmd_handler(httpd_req_t *req){
    char*  buf;
    size_t buf_len;
    char variable[32] = {0,};
    char value[32] = {0,};
```

```
    buf_len = httpd_req_get_url_query_len(req) + 1;
    if (buf_len > 1) {
        buf = (char*)malloc(buf_len);
        if(!buf){
            httpd_resp_send_500(req);
            return ESP_FAIL;
        }
        if (httpd_req_get_url_query_str(req, buf, buf_len) == ESP_OK) {
            if (httpd_query_key_value(buf, "var", variable, sizeof(variable)) == ESP_OK
&&
                httpd_query_key_value(buf, "val", value, sizeof(value)) == ESP_OK) {
            } else {
                free(buf);
                httpd_resp_send_404(req);
                return ESP_FAIL;
            }
        } else {
            free(buf);
            httpd_resp_send_404(req);
            return ESP_FAIL;
        }
        free(buf);
    } else {
        httpd_resp_send_404(req);
        return ESP_FAIL;
    }
```

```
    int val = atoi(value);
    sensor_t * s = esp_camera_sensor_get();
    int res = 0;

    if(!strcmp(variable, "framesize")) {
        if(s->pixformat == PIXFORMAT_JPEG) res = s->set_framesize(s,
(framesize_t)val);
    }
    else if(!strcmp(variable, "quality")) res = s->set_quality(s, val);
    else if(!strcmp(variable, "contrast")) res = s->set_contrast(s, val);
    else if(!strcmp(variable, "brightness")) res = s->set_brightness(s, val);
    else if(!strcmp(variable, "saturation")) res = s->set_saturation(s, val);
    else if(!strcmp(variable, "gainceiling")) res = s->set_gainceiling(s,
(gainceiling_t)val);
    else if(!strcmp(variable, "colorbar")) res = s->set_colorbar(s, val);
    else if(!strcmp(variable, "awb")) res = s->set_whitebal(s, val);
    else if(!strcmp(variable, "agc")) res = s->set_gain_ctrl(s, val);
    else if(!strcmp(variable, "aec")) res = s->set_exposure_ctrl(s, val);
    else if(!strcmp(variable, "hmirror")) res = s->set_hmirror(s, val);
    else if(!strcmp(variable, "vflip")) res = s->set_vflip(s, val);
    else if(!strcmp(variable, "awb_gain")) res = s->set_awb_gain(s, val);
    else if(!strcmp(variable, "agc_gain")) res = s->set_agc_gain(s, val);
    else if(!strcmp(variable, "aec_value")) res = s->set_aec_value(s, val);
    else if(!strcmp(variable, "aec2")) res = s->set_aec2(s, val);
    else if(!strcmp(variable, "dcw")) res = s->set_dcw(s, val);
    else if(!strcmp(variable, "bpc")) res = s->set_bpc(s, val);
    else if(!strcmp(variable, "wpc")) res = s->set_wpc(s, val);
    else if(!strcmp(variable, "raw_gma")) res = s->set_raw_gma(s, val);
```

```
    else if(!strcmp(variable, "lenc")) res = s->set_lenc(s, val);
    else if(!strcmp(variable, "special_effect")) res = s->set_special_effect(s, val);
    else if(!strcmp(variable, "wb_mode")) res = s->set_wb_mode(s, val);
    else if(!strcmp(variable, "ae_level")) res = s->set_ae_level(s, val);
    else {
        res = -1;
    }

    if(res){
        return httpd_resp_send_500(req);
    }

    httpd_resp_set_hdr(req, "Access-Control-Allow-Origin", "*");
    return httpd_resp_send(req, NULL, 0);
}

static esp_err_t status_handler(httpd_req_t *req){
    static char json_response[1024];

    sensor_t * s = esp_camera_sensor_get();
    char * p = json_response;
    *p++ = '{';

    p+=sprintf(p, "\"framesize\":%u,", s->status.framesize);
    p+=sprintf(p, "\"quality\":%u,", s->status.quality);
    p+=sprintf(p, "\"brightness\":%d,", s->status.brightness);
    p+=sprintf(p, "\"contrast\":%d,", s->status.contrast);
    p+=sprintf(p, "\"saturation\":%d,", s->status.saturation);
```

```
    p+=sprintf(p, "\"special_effect\":%u,", s->status.special_effect);
    p+=sprintf(p, "\"wb_mode\":%u,", s->status.wb_mode);
    p+=sprintf(p, "\"awb\":%u,", s->status.awb);
    p+=sprintf(p, "\"awb_gain\":%u,", s->status.awb_gain);
    p+=sprintf(p,  "\"aec\":%u,",  s->status.aec);
    p+=sprintf(p, "\"aec2\":%u,", s->status.aec2);
    p+=sprintf(p, "\"ae_level\":%d,", s->status.ae_level);
    p+=sprintf(p, "\"aec_value\":%u,", s->status.aec_value);
    p+=sprintf(p, "\"agc\":%u,", s->status.agc);
    p+=sprintf(p, "\"agc_gain\":%u,", s->status.agc_gain);
    p+=sprintf(p, "\"gainceiling\":%u,", s->status.gainceiling);
    p+=sprintf(p, "\"bpc\":%u,", s->status.bpc);
    p+=sprintf(p, "\"wpc\":%u,", s->status.wpc);
    p+=sprintf(p, "\"raw_gma\":%u,", s->status.raw_gma);
    p+=sprintf(p, "\"lenc\":%u,", s->status.lenc);
    p+=sprintf(p, "\"hmirror\":%u,", s->status.hmirror);
    p+=sprintf(p, "\"dcw\":%u,", s->status.dcw);
    p+=sprintf(p, "\"colorbar\":%u", s->status.colorbar);
    *p++ = '}';
    *p++ = 0;
    httpd_resp_set_type(req, "application/json");
    httpd_resp_set_hdr(req, "Access-Control-Allow-Origin", "*");
    return httpd_resp_send(req, json_response, strlen(json_response));
}

static esp_err_t index_handler(httpd_req_t *req){
    httpd_resp_set_type(req, "text/html");
    String page = "";
```

```
    page += "<meta name=\"viewport\" content=\"width=device-width, initial-
scale=1.0, maximum-scale=1.0, user-scalable=0\">\n";
 page += "<script>var xhttp = new XMLHttpRequest();</script>";
 page += "<script>function getsend(arg) { xhttp.open('GET', arg +'?' + new
Date().getTime(), true); xhttp.send() } </script>";
 //page += "<p align=center><IMG SRC='http://" + WiFiAddr + ":81/stream'
style='width:280px;'></p><br/><br/>";
 page += "<p align=center><IMG SRC='http://" + WiFiAddr + ":81/stream'
style='width:300px; transform:rotate(180deg);'></p><br/><br/>";

 page += "<p align=center> <button style=background-
color:cyan;width:90px;height:80px onmousedown=getsend('go')
onmouseup=getsend('stop') ontouchstart=getsend('go') ontouchend=getsend('stop')
><b>Forward</b></button> </p>";
 page += "<p align=center>";
 page += "<button style=background-color:cyan;width:90px;height:80px;
onmousedown=getsend('left') onmouseup=getsend('stop') ontouchstart=getsend('left')
ontouchend=getsend('stop')><b>Left</b></button> ";
 page += "<button style=background-color:lightgray;width:90px;height:80px
onmousedown=getsend('stop')
onmouseup=getsend('stop')><b>Stop</b></button> ";
 page += "<button style=background-color:cyan;width:90px;height:80px
onmousedown=getsend('right') onmouseup=getsend('stop')
ontouchstart=getsend('right') ontouchend=getsend('stop')><b>Right</b></button>";
page += "</p>";

 page += "<p align=center><button style=background-
color:cyan;width:90px;height:80px onmousedown=getsend('back')
```

```
onmouseup=getsend('stop') ontouchstart=getsend('back') ontouchend=getsend('stop')
><b>Backward</b></button></p>";

 page += "<p align=center>";
 page += "<button style=background-color:yellow;width:140px;height:40px
onmousedown=getsend('ledon')><b>Light ON</b></button>";
 page += "<button style=background-color:yellow;width:140px;height:40px
onmousedown=getsend('ledoff')><b>Light OFF</b></button>";
 page += "</p>";

    return httpd_resp_send(req, &page[0], strlen(&page[0]));
}

static esp_err_t go_handler(httpd_req_t *req){
    WheelAct(HIGH, LOW, HIGH, LOW);
    Serial.println("Go");
    httpd_resp_set_type(req, "text/html");
    return httpd_resp_send(req, "OK", 2);
}
static esp_err_t back_handler(httpd_req_t *req){
    WheelAct(LOW, HIGH, LOW, HIGH);
    Serial.println("Back");
    httpd_resp_set_type(req, "text/html");
    return httpd_resp_send(req, "OK", 2);
}

static esp_err_t left_handler(httpd_req_t *req){
    WheelAct(LOW, HIGH, HIGH, LOW);
```

```
    Serial.println("Left");
    httpd_resp_set_type(req, "text/html");
    return httpd_resp_send(req, "OK", 2);
}
static esp_err_t right_handler(httpd_req_t *req){
    WheelAct(HIGH, LOW, LOW, HIGH);
    Serial.println("Right");
    httpd_resp_set_type(req, "text/html");
    return httpd_resp_send(req, "OK", 2);
}

static esp_err_t stop_handler(httpd_req_t *req){
    WheelAct(LOW, LOW, LOW, LOW);
    Serial.println("Stop");
    httpd_resp_set_type(req, "text/html");
    return httpd_resp_send(req, "OK", 2);
}

static esp_err_t ledon_handler(httpd_req_t *req){
    digitalWrite(gpLed, HIGH);
    Serial.println("LED ON");
    httpd_resp_set_type(req, "text/html");
    return httpd_resp_send(req, "OK", 2);
}
static esp_err_t ledoff_handler(httpd_req_t *req){
    digitalWrite(gpLed, LOW);
    Serial.println("LED OFF");
    httpd_resp_set_type(req, "text/html");
```

```
    return httpd_resp_send(req, "OK", 2);
}

void startCameraServer(){
    httpd_config_t config = HTTPD_DEFAULT_CONFIG();

    httpd_uri_t go_uri = {
        .uri       = "/go",
        .method    = HTTP_GET,
        .handler   = go_handler,
        .user_ctx  = NULL
    };

    httpd_uri_t back_uri = {
        .uri       = "/back",
        .method    = HTTP_GET,
        .handler   = back_handler,
        .user_ctx  = NULL
    };

    httpd_uri_t stop_uri = {
        .uri       = "/stop",
        .method    = HTTP_GET,
        .handler   = stop_handler,
        .user_ctx  = NULL
    };

    httpd_uri_t left_uri = {
```

```
    .uri       = "/left",
    .method    = HTTP_GET,
    .handler   = left_handler,
    .user_ctx  = NULL
};

httpd_uri_t right_uri = {
    .uri       = "/right",
    .method    = HTTP_GET,
    .handler   = right_handler,
    .user_ctx  = NULL
};

httpd_uri_t ledon_uri = {
    .uri       = "/ledon",
    .method    = HTTP_GET,
    .handler   = ledon_handler,
    .user_ctx  = NULL
};

httpd_uri_t ledoff_uri = {
    .uri       = "/ledoff",
    .method    = HTTP_GET,
    .handler   = ledoff_handler,
    .user_ctx  = NULL
};

httpd_uri_t index_uri = {
```

```
    .uri       = "/",
    .method    = HTTP_GET,
    .handler   = index_handler,
    .user_ctx  = NULL
};

httpd_uri_t status_uri = {
    .uri       = "/status",
    .method    = HTTP_GET,
    .handler   = status_handler,
    .user_ctx  = NULL
};

httpd_uri_t cmd_uri = {
    .uri       = "/control",
    .method    = HTTP_GET,
    .handler   = cmd_handler,
    .user_ctx  = NULL
};

httpd_uri_t capture_uri = {
    .uri       = "/capture",
    .method    = HTTP_GET,
    .handler   = capture_handler,
    .user_ctx  = NULL
};

httpd_uri_t stream_uri = {
```

```
        .uri       = "/stream",
        .method    = HTTP_GET,
        .handler   = stream_handler,
        .user_ctx  = NULL
    };

    ra_filter_init(&ra_filter, 20);
    Serial.printf("Starting web server on port: '%d'", config.server_port);
    if (httpd_start(&camera_httpd, &config) == ESP_OK) {
        httpd_register_uri_handler(camera_httpd, &index_uri);
        httpd_register_uri_handler(camera_httpd, &go_uri);
        httpd_register_uri_handler(camera_httpd, &back_uri);
        httpd_register_uri_handler(camera_httpd, &stop_uri);
        httpd_register_uri_handler(camera_httpd, &left_uri);
        httpd_register_uri_handler(camera_httpd, &right_uri);
        httpd_register_uri_handler(camera_httpd, &ledon_uri);
        httpd_register_uri_handler(camera_httpd, &ledoff_uri);
    }

    config.server_port += 1;
    config.ctrl_port += 1;
    Serial.printf("Starting stream server on port: '%d'", config.server_port);
    if (httpd_start(&stream_httpd, &config) == ESP_OK) {
        httpd_register_uri_handler(stream_httpd, &stream_uri);
    }
}

void WheelAct(int nLf, int nLb, int nRf, int nRb)
```

```
{
 digitalWrite(gpLf, nLf);
 digitalWrite(gpLb, nLb);
 digitalWrite(gpRf, nRf);
 digitalWrite(gpRb,  nRb);
}
```

A.2 ESP32 Camera Surveillance Car Code

```
#include "esp_camera.h"
#include <WiFi.h>

//
// WARNING!!! Make sure that you have either selected ESP32 Wrover Module,
//            or another board which has PSRAM enabled
//
// Adafruit ESP32 Feather

// Select camera model
//#define CAMERA_MODEL_WROVER_KIT
//#define CAMERA_MODEL_M5STACK_PSRAM
#define CAMERA_MODEL_AI_THINKER

const char *ssid = "TRI_Asianet_2.4G";
const char *password = "200C86B23360";

IPAddress local_IP(192, 168, 1, 92);      // Static IP
IPAddress gateway(192, 168, 1, 1);        // Your router gateway
IPAddress subnet(255, 255, 255, 0);       // Subnet mask
```

```
IPAddress primaryDNS(8, 8, 8, 8);         // Optional
IPAddress secondaryDNS(8, 8, 4, 4);       // Optional

#if defined(CAMERA_MODEL_WROVER_KIT)
#define PWDN_GPIO_NUM     -1
#define RESET_GPIO_NUM    -1
#define XCLK_GPIO_NUM     21
#define SIOD_GPIO_NUM     26
#define SIOC_GPIO_NUM     27

#define Y9_GPIO_NUM       35
#define Y8_GPIO_NUM       34
#define Y7_GPIO_NUM       39
#define Y6_GPIO_NUM       36
#define Y5_GPIO_NUM       19
#define Y4_GPIO_NUM       18
#define Y3_GPIO_NUM        5
#define Y2_GPIO_NUM        4
#define VSYNC_GPIO_NUM    25
#define HREF_GPIO_NUM     23
#define PCLK_GPIO_NUM     22

#elif defined(CAMERA_MODEL_AI_THINKER)
#define PWDN_GPIO_NUM      32
#define RESET_GPIO_NUM     -1
#define XCLK_GPIO_NUM       0
#define SIOD_GPIO_NUM      26
#define SIOC_GPIO_NUM      27
```

```
#define Y9_GPIO_NUM       35
#define Y8_GPIO_NUM       34
#define Y7_GPIO_NUM       39
#define Y6_GPIO_NUM       36
#define Y5_GPIO_NUM       21
#define Y4_GPIO_NUM       19
#define Y3_GPIO_NUM       18
#define Y2_GPIO_NUM        5
#define VSYNC_GPIO_NUM    25
#define HREF_GPIO_NUM     23
#define PCLK_GPIO_NUM     22

#else
#error "Camera model not selected"
#endif

// GPIO Setting
extern int gpLb =  2; // Left 1
extern int gpLf = 14; // Left 2
extern int gpRb = 15; // Right 1
extern int gpRf = 13; // Right 2
extern int gpLed =  4; // Light
extern String WiFiAddr ="";

void startCameraServer();

void setup() {
```

```
Serial.begin(115200);
Serial.setDebugOutput(true);
Serial.println();

pinMode(gpLb, OUTPUT); //Left Backward
pinMode(gpLf, OUTPUT); //Left Forward
pinMode(gpRb, OUTPUT); //Right Forward
pinMode(gpRf, OUTPUT); //Right Backward
pinMode(gpLed, OUTPUT); //Light

//initialize
digitalWrite(gpLb, LOW);
digitalWrite(gpLf, LOW);
digitalWrite(gpRb, LOW);
digitalWrite(gpRf, LOW);
digitalWrite(gpLed, LOW);

camera_config_t config;
config.ledc_channel = LEDC_CHANNEL_0;
config.ledc_timer = LEDC_TIMER_0;
config.pin_d0 = Y2_GPIO_NUM;
config.pin_d1 = Y3_GPIO_NUM;
config.pin_d2 = Y4_GPIO_NUM;
config.pin_d3 = Y5_GPIO_NUM;
config.pin_d4 = Y6_GPIO_NUM;
config.pin_d5 = Y7_GPIO_NUM;
config.pin_d6 = Y8_GPIO_NUM;
config.pin_d7 = Y9_GPIO_NUM;
```

```
config.pin_xclk = XCLK_GPIO_NUM;
config.pin_pclk = PCLK_GPIO_NUM;
config.pin_vsync = VSYNC_GPIO_NUM;
config.pin_href = HREF_GPIO_NUM;
config.pin_sscb_sda = SIOD_GPIO_NUM;
config.pin_sscb_scl = SIOC_GPIO_NUM;
config.pin_pwdn = PWDN_GPIO_NUM;
config.pin_reset = RESET_GPIO_NUM;
config.xclk_freq_hz = 20000000;
config.pixel_format = PIXFORMAT_JPEG;
//init with high specs to pre-allocate larger buffers
if(psramFound()){
  config.frame_size = FRAMESIZE_UXGA;
  config.jpeg_quality = 10;
  config.fb_count = 2;
} else {
  config.frame_size = FRAMESIZE_SVGA;
  config.jpeg_quality = 12;
  config.fb_count = 1;
}

// camera init
esp_err_t err = esp_camera_init(&config);
if (err != ESP_OK) {
  Serial.printf("Camera init failed with error 0x%x", err);
  return;
}
```

```
  //drop down frame size for higher initial frame rate
  sensor_t * s = esp_camera_sensor_get();
  s->set_framesize(s, FRAMESIZE_CIF);
  s->set_vflip(s, 1);

  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("");
  Serial.println("WiFi connected");

  startCameraServer();

  Serial.print("Camera Ready! Use 'http://");
  Serial.print(WiFi.localIP());
  WiFiAddr = WiFi.localIP().toString();
  Serial.println("' to connect");
}

void loop() {
  // put your main code here, to run repeatedly:

}
```